

Predicting Washington D.C. Residential Property Prices Using Deep Neural Networks with H2O and LIME Interpretability

Mohammad Hadi Yaqoobi

*Cornell University — AI360 Neural Networks and Machine Learning
IT Business Systems Analyst | Boston, MA*

Abstract

This paper presents the design, training, and evaluation of a deep neural network model for predicting residential property sale prices in Washington D.C. using the H2O deep learning framework in R. The dataset comprised 10,617 residential property transactions with 108 features spanning structural attributes, geographic indicators, neighborhood dummy variables, and seasonal covariates. After preprocessing and correlation-guided feature selection, a three-hidden-layer neural network ($64 \rightarrow 32 \rightarrow 16$ nodes) using ReLU activation was trained on 70% of the data and validated on the remaining 30%. The model achieved a validation R^2 of 0.811 and a dollar-scale mean absolute error of approximately \$73,644. Local Interpretable Model-agnostic Explanations (LIME) were applied to five held-out observations, revealing that effective year built, geographic quadrant, ward classification, and gross building area were the dominant price drivers.

I. INTRODUCTION

Accurate real estate valuation is a problem of significant economic importance, affecting buyers, sellers, mortgage lenders, and public policy alike. Traditional hedonic pricing models rely on linear regression frameworks that assume additive, independent contributions from property characteristics — an assumption that routinely fails to capture the nonlinear interactions between structural quality, neighborhood prestige, and market timing that characterize real estate markets.

Deep neural networks offer a compelling alternative: they can learn arbitrary nonlinear mappings from high-dimensional feature spaces without requiring hand-crafted interaction terms. The H2O open-source machine learning platform provides a production-grade implementation in R with built-in distributed training, adaptive learning rates, and L1/L2 regularization suited to tabular real estate data.

However, the black-box nature of neural networks creates a critical interpretability gap in high-stakes valuation contexts. To bridge this gap, we apply the LIME framework (Ribeiro et al., 2016), which approximates the complex model locally around each individual prediction with a simple linear surrogate, identifying which features most strongly influenced each valuation outcome.

The remainder of the paper is organized as follows. Section II describes the dataset and preprocessing pipeline. Section III details the neural network architecture and training methodology. Section IV provides the complete R code. Section V presents results, variable importance rankings, and LIME analysis. Section VI concludes with directions for future work.

II. DATASET AND PREPROCESSING

A. Data Source

The dataset is the D.C. Residential Properties dataset published by ChrisC on Kaggle (2018), containing residential property sale transactions in Washington D.C. priced under \$1,000,000. The raw dataset contains 10,617 observations across 108 columns including structural attributes, geographic region indicators, neighborhood-level dummy variables, ward classifications, and seasonal sale-timing variables.

B. Target Variable

The target variable is logPrice — the base-10 logarithm of the sale price in dollars. Log transformation was applied because raw sale prices exhibit strong positive skew. The log scale compresses extreme values, produces a more Gaussian-distributed target, and reduces the disproportionate influence of very high-value outliers on the quadratic loss used during training. Predictions were unscaled to dollar terms (10^{logPrice}) for final error reporting.

C. Missing Values

A column-wise missing value audit revealed at most 21 missing observations per column out of 10,617 total rows — a missingness rate below 0.2%. Rows containing any missing values were removed via `na.omit()`, yielding a clean dataset of 10,596 observations with no imputation required.

D. Feature Selection

Pearson correlation coefficients between all numeric features and logPrice were computed and sorted. Variables with near-zero correlations ($|r| < 0.02$), pure identifier columns (FIPS codes, census tracts, parcel squares), geographic coordinates, and the raw PRICE column were excluded. The final model used 35 features across five categories, summarized in Table I.

Category	Variables Included
Structural	GrossBuildingArea, BATHROOMS, BATHRM, HF_BATHRM, BEDRM, ROOMS, FIREPLACES, KITCHENS, STORIES, LANDAREA, AC1IsYes, EYB
Property Type	RowInside, RowEnd, SemiDetached, Multi
Geographic	NW quadrant, Ward2, Ward3, Ward4, Ward5, Ward6
Neighborhood	ChevyChase, Georgetown, CapitolHill, ColumbiaHeights, OldCity1, OldCity2, CongressHeights, Deanwood, Anacostia
Seasonal	Winter, Spring, Summer
Grade	GradeLowerBetter

TABLE I — Feature categories and variables used in the final model.

III. METHODOLOGY

A. Train / Test Split

The cleaned dataset was randomly partitioned into a training set (70%, $n = 7,417$) and a held-out test set (30%, $n = 3,179$) using `set.seed(42)` for reproducibility. Both partitions were converted to H2O distributed data frames via `as.h2o()`. An additional random sample of five observations was drawn from the training set to serve as the LIME explanation corpus.

B. Neural Network Architecture

The model was constructed using `h2o.deeplearning()` with the configuration detailed in Table II. The progressively narrowing hidden layer structure ($64 \rightarrow 32 \rightarrow 16$) encourages the network to build increasingly abstract representations at each successive layer before producing the scalar price prediction at the linear output neuron.

Hyperparameter	Value	Rationale
Hidden layers	3 ($64 \rightarrow 32 \rightarrow 16$ nodes)	Hierarchical feature abstraction
Activation function	Rectifier (ReLU)	Avoids vanishing gradients; regression-friendly
Adaptive learning rate	TRUE (ADADELTA)	Automatic rate decay for stable convergence
L1 regularization	0.0001	Mild weight sparsity
L2 regularization	0.001	Dominant weight shrinkage; controls overfitting
Epochs	50	Full convergence without excessive runtime
Input standardization	TRUE	Zero-mean, unit-variance normalization
Random seed	42	Full reproducibility

TABLE II — Neural network hyperparameter configuration.

C. Regularization Strategy

Both L1 (lasso) and L2 (ridge) regularization were applied simultaneously. With L2 set ten times larger than L1 (0.001 vs. 0.0001), the algorithm favored weight shrinkage over sparsity — producing a dense but small-weighted network rather than one with many zero weights. This is appropriate for a regression task where all 35 features carry at least some predictive signal, and it directly contributed to the small training-to-validation performance gap observed in the results.

D. LIME Interpretability

Post-hoc local explanations were generated using the `lime` R package. The `lime()` function constructs a model-agnostic explainer trained on the full training data distribution. For each of the five sampled observations, `explain()` perturbs the input features and fits a local linear surrogate model, returning the six features most responsible for each individual prediction. Results were visualized using `plot_features()` and `plot_explanations()`.

IV. COMPLETE R CODE

A. Environment Setup and Data Loading

```
library(h2o)
library(lime)

# Load dataset
dc_data <- read.csv("DC_PropertyResidentialunder1mill.csv")
dim(dc_data)      # 10617 x 108
names(dc_data)
head(dc_data)
summary(dc_data)

# Missing value audit
missing <- colSums(is.na(dc_data))
missing[missing > 0]

# Correlation analysis with logPrice
num_cols <- dc_data[, sapply(dc_data, is.numeric)]
num_cols <- num_cols[, !names(num_cols) %in%
  c("X.1", "PRICE", "CENSUS_TRACT", "CENSUS_BLOCK", "SQUARE", "X", "Y")]
cor_logPrice <- cor(num_cols, use = "complete.obs")["logPrice", ]
sort(cor_logPrice, decreasing = TRUE)
```

B. Feature Selection and Cleaning

```
dc_clean <- na.omit(dc_data)
cat("Rows after NA removal:", nrow(dc_clean), "\n") # 10596

keep_cols <- c(
  "logPrice", # TARGET
  "GrossBuildingArea", "BATHROOMS", "BATHRM",
  "FIREPLACES", "BEDRM", "ROOMS",
  "AC1IsYes", "HF_BATHRM", "KITCHENS",
  "EYB", "STORIES", "GradeLowerBetter",
  "LANDAREA", "NW",
  "Ward2", "Ward3", "Ward4", "Ward5", "Ward6",
  "RowInside", "RowEnd", "SemiDetached", "Multi",
  "Winter", "Spring", "Summer",
  "ChevyChase", "Georgetown", "CapitolHill",
  "ColumbiaHeights", "OldCity1", "OldCity2",
  "CongressHeights", "Deanwood", "Anacostia",
  "Residential_0.Condo_1"
)

model_data <- dc_clean[, keep_cols]
cat("Final dimensions:", dim(model_data), "\n") # 10596 x 37
```

C. Train / Test Split and LIME Sample Selection

```
set.seed(42)
n <- nrow(model_data)
train_index <- sample(1:n, size = round(0.70 * n), replace = FALSE)
train_df <- model_data[train_index, ] # 7417 rows
test_df <- model_data[-train_index, ] # 3179 rows

cat("Training observations:", nrow(train_df), "\n")
cat("Test observations: ", nrow(test_df), "\n")

# Select 5 training observations for LIME analysis
set.seed(42)
```

```
lime_rows      <- sample(1:nrow(train_df), 5)
data_for_lime <- train_df[lime_rows, ]
print(data_for_lime[, c("logPrice", "GrossBuildingArea", "BATHROOMS", "BEDRM", "NW")])
```

D. H2O Initialization and Frame Conversion

```
h2o.init(max_mem_size = "2g")

train_h2o <- as.h2o(train_df)
test_h2o  <- as.h2o(test_df)

cat("Train H2O frame:", nrow(train_h2o), "rows\n")
cat("Test  H2O frame:", nrow(test_h2o),  "rows\n")
```

E. Deep Neural Network Training

```
target      <- "logPrice"
predictors  <- setdiff(names(train_h2o), target)

set.seed(42)
dl_model <- h2o.deeplearning(
  y          = target,
  x          = predictors,
  training_frame = train_h2o,
  validation_frame = test_h2o,
  hidden      = c(64, 32, 16),    # three hidden layers
  activation   = "Rectifier",     # ReLU
  adaptive_rate = TRUE,           # ADADELTA adaptive LR
  l1           = 0.0001,          # L1 regularization
  l2           = 0.001,           # L2 regularization
  epochs       = 50,
  standardize  = TRUE,
  seed         = 42
)

summary(dl_model)          # full model summary
plot(dl_model)             # training vs. validation loss curve
h2o.varimp_plot(dl_model)  # variable importance histogram
```

F. Predictions and Error Metrics

```
test_pred      <- h2o.predict(dl_model, test_h2o)
test_pred_df   <- as.data.frame(test_pred)
test_actual    <- as.data.frame(test_h2o$logPrice)

# Log-scale error metrics
mae_test <- mean(abs(test_pred_df$predict - test_actual$logPrice))
rmse_test <- sqrt(mean((test_pred_df$predict - test_actual$logPrice)^2))
cat("Test MAE (log scale):", round(mae_test, 4), "\n")
cat("Test RMSE (log scale):", round(rmse_test, 4), "\n")

# Dollar-scale MAE
pred_dollars <- 10^test_pred_df$predict
actual_dollars <- 10^test_actual$logPrice
mae_dollars <- mean(abs(pred_dollars - actual_dollars))
cat("Test MAE (dollars): $", round(mae_dollars, 0), "\n")

# Predicted vs. Actual scatter plot
plot(actual_dollars, pred_dollars,
  xlab = "Actual Sale Price ($)",
  ylab = "Predicted Sale Price ($)",
  main = "Predicted vs. Actual House Prices – Test Set",
```

```
pch = 16, cex = 0.4, col = "grey40")
abline(0, 1, col = "black", lwd = 1.5, lty = 2)
```

G. LIME Interpretability Analysis

```
# Predict on LIME sample
lime_h2o <- as.h2o(data_for_lime)
lime_preds <- h2o.predict(dl_model, lime_h2o)
cat("LIME predictions (logPrice):\n")
print(as.data.frame(lime_preds))
cat("Actual logPrice values:\n")
print(data_for_lime$logPrice)

# Build explainer on training data
explainer <- lime(train_df, dl_model)

# Generate local explanations (6 features per observation)
explanation <- explain(data_for_lime, explainer, n_labels = 1, n_features = 6)

# Visualize
plot_features(explanation, ncol = 2)
plot_explanations(explanation)
```

V. RESULTS AND DISCUSSION

A. Model Performance

H2O completed training in approximately 8.8 seconds across 50 epochs, processing 370,850 training samples. The scoring history confirmed steady convergence: training RMSE fell from 0.0761 at epoch 5 to 0.0669 at epoch 50, while validation RMSE stabilized at approximately 0.0790 from epoch 5 onward — indicating that the model reached near-optimal generalization early in training. Table III summarizes the final performance metrics on both partitions.

Metric	Training Set	Test Set
MSE	0.004471	0.006236
RMSE	0.06687	0.07897
MAE (log scale)	0.04791	0.05835
RMSLE	0.01010	0.01188
R ²	0.868	0.811
MAE (dollar scale)	—	\$73,644

TABLE III — Model performance metrics on training and test partitions.

The model explained 81.1% of variance in house prices on unseen test data ($R^2 = 0.811$). The dollar-scale MAE of \$73,644 represents roughly 7–10% of the median DC property price, indicating practically useful accuracy for mid-market properties. The modest gap between training R^2 (0.868) and test R^2 (0.811) confirms that the L2-dominant regularization strategy effectively controlled overfitting across all three hidden layers.

B. Variable Importance

The `h2o.varimp_plot()` output ranked all input features by their relative contribution to model predictions. Table IV presents the ten most important variables identified by H2O's internal weight-based importance scoring.

Rank	Variable	Rel. Importance	Pct. Share
1	EYB (Effective Year Built)	1.000	5.63%
2	NW (Northwest Quadrant)	0.862	4.85%
3	Ward5	0.802	4.51%
4	Ward4	0.751	4.23%
5	GrossBuildingArea	0.713	4.01%
6	BATHROOMS	~0.65	~3.7%
7	BEDRM	~0.60	~3.4%
8	FIREPLACES	~0.58	~3.3%
9	GradeLowerBetter	~0.55	~3.1%
10	ROOMS	~0.50	~2.8%

TABLE IV — Top ten variables by relative importance score.

Effective year built (EYB) emerged as the single most important predictor, reflecting the strong premium DC buyers place on newer construction and recent renovation. The NW quadrant indicator ranked second, capturing Washington D.C.'s pronounced geographic price divide: Northwest neighborhoods such as Georgetown, Chevy Chase, and Cleveland Park command values several times higher than comparable properties in Southeast or Northeast quadrants. Ward-level indicators (Ward4, Ward5) further decomposed geographic price variation at finer administrative resolution. GrossBuildingArea ranking fifth reinforces the fundamental real estate principle that physical size is a primary value driver independent of location.

C. LIME Analysis

LIME explanations were generated for five sampled training properties. Table V compares predicted versus actual logPrice values and summarizes the dominant LIME feature drivers for each observation.

Obs.	Actual logPrice	Predicted logPrice	Error	Primary LIME Drivers
1	5.423	5.459	+0.036	EYB (+), Ward (-), GrossBuildingArea (+)
2	5.439	5.430	-0.009	NW (-), BATHROOMS (-), EYB (+)
3	5.832	5.943	+0.111	NW=1 (+), GrossBuildingArea (+)
4	5.004	5.581	+0.577	Conflicting ward signals; EYB overestimate
5	5.717	5.824	+0.107	BEDRM (+), GrossBuildingArea (+), Ward (+)

TABLE V — LIME sample: predicted vs. actual values with key feature drivers.

Across all five explanations, EYB and NW quadrant appeared as consistent top contributors — directly mirroring their global variable importance rankings and providing reassurance that the model's internal representations align with known real estate economics. Observation 4 exhibited the largest prediction error (0.577 log units), where conflicting ward-level signals caused the model to substantially overestimate a lower-priced property. This case highlights the model's known weakness at the lower end of the price distribution, where training examples are fewer and feature signals are less consistent.

VI. CONCLUSION AND FUTURE WORK

This paper demonstrated the effective application of regularized deep learning to residential property valuation in Washington D.C. The H2O three-layer neural network ($64 \rightarrow 32 \rightarrow 16$, ReLU, ADADELTA, L1/L2 regularization) achieved a test R^2 of 0.811 and a dollar-scale MAE of approximately \$73,644 on 3,179 held-out properties — explaining 81% of price variance through structural, geographic, and demographic features. The LIME post-hoc analysis confirmed that effective year built, NW quadrant designation, and gross building area were the dominant value drivers, consistent with established Washington D.C. real estate economics.

The close training-to-validation performance gap (R^2 of 0.868 vs. 0.811) confirms that the regularization strategy was appropriate for this dataset. However, the model showed meaningful weakness for lower-priced properties and atypical observations where geographic feature signals conflict. Four directions for future improvement are identified:

- **Systematic hyperparameter search.** Using `h2o.grid()` to exhaustively search layer sizes, activation functions, dropout rates, and regularization coefficients would likely identify an architecture superior to the manually configured model.

- **External feature enrichment.** Adding Metro station proximity, school quality ratings, walkability scores, and neighborhood crime statistics could meaningfully reduce the \$73,644 MAE by capturing location-quality dimensions absent from the current feature set.
- **Ensemble modeling.** Combining the neural network with a gradient boosted model (`h2o.gbm()`) in a stacked ensemble would leverage complementary strengths — gradient boosting excels at structured tabular data while the neural network captures higher-order interactions.
- **Temporal validation.** Training on earlier transactions and testing on more recent sales would provide a more realistic assessment of generalization to future market conditions, given the sensitivity of DC real estate to interest rate policy and neighborhood development cycles.

Deep learning combined with LIME interpretability represents a promising direction for automated valuation models (AVMs) that are both accurate and explainable — critical properties for deployment in regulated lending and appraisal contexts.

. REFERENCES

- [1] C. Correa, "D.C. Residential Properties," Kaggle, 2018. [Online]. Available: <https://www.kaggle.com/christophercorrea/dc-residential-properties>
- [2] H2O.ai, "H2O Deep Learning — User Guide," H2O Documentation, 2023. [Online]. Available: <https://docs.h2o.ai>
- [3] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why Should I Trust You?': Explaining the Predictions of Any Classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, 2016, pp. 1135–1144.
- [4] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY: Springer, 2009.
- [5] R Core Team, "R: A Language and Environment for Statistical Computing," R Foundation for Statistical Computing, Vienna, Austria, 2020.
- [6] T. K. Peterson and C. Ziegler, "lime: Local Interpretable Model-Agnostic Explanations," R package version 0.5.2, CRAN, 2021.